

Table 2: AI's role across the software development life cycle

SDLC phase	AI-driven impact	Examples/Benefits
Planning and ideation	Translates ideas into structured requirements and technical specs.	Generate user stories, architecture outlines, or flowcharts from prompts.
Development	Speeds up coding with autocompletion, code snippets, and boilerplate generation.	Build features faster, reduce manual coding, accelerate MVP delivery.
Testing	Auto-generates unit tests, suggests edge cases, and simulates input scenarios.	Improve test coverage and catch bugs early in development.
Code review and debugging	Flags issues, explains code, and suggests optimisations.	Faster, more inclusive reviews; easier onboarding for junior developers.
Deployment and DevOps	Assists with infrastructure-as-code and script generation for CI/CD.	Generate Dockerfiles, Kubernetes manifests, or pipeline configs from text prompts.
Maintenance	Summarises legacy code, auto-updates documentation, and flags outdated patterns.	Reduces technical debt, improves long-term maintainability.
Collaboration	Bridges gaps between technical and non-technical teams with summaries and visuals.	Converts code to readable insights, enabling clearer stakeholder communication.



Figure 3: Impact of GenAI on SDLC

Generation of test cases: The writing of test cases is an important task in maintaining software, though it is time-consuming and tedious. GenAI automates it by generating unit tests of your current functions and classes. By examining method signatures and the code logic, it suggests relevant test scenarios, edge cases, and even mock data.

This not only improves test coverage but also catches bugs earlier in the development lifecycle, resulting in more stable and reliable applications.

Refactoring and code explanation: Refactoring is the art of cleaning up

existing code while keeping functionality intact—a feat at which GenAI excels. It suggests better names for variables, corrects logic that is unnecessarily complex, or transforms old paradigms into newer and better ones.

AI can also, in plain language, describe what a piece of code does. This ability aids in the understanding of codebases inherited by team members, in the onboarding of new developers, or in the documentation of complex functions. This enhances teamwork and makes long-term maintenance easier.

Generative AI is also radically changing the entire software

development life cycle (SDLC) itself — from formulating a product idea to its deployment and long-term maintenance. By automating repetitive tasks, improving code quality, and simplifying processes, teams can move fast and build more resilient software. Table 2 shows how GenAI affects every important phase of the SDLC

Challenges and ethical considerations

The promise of generative AI in programming is immense; however, it does come with a series of challenges and ethical considerations that cannot be ignored. Developers must be vigilant about addressing these concerns as long as these tools continue to evolve, so that AI can be applied to the software development life cycle in a responsible and effective manner.

Code quality and reliability: One of the greatest challenges of using generative AI for programming concerns the quality and reliability of the code generated. Although AI-generated code often satisfies the syntactical criteria, it may not necessarily adhere to best practices or be logically sound. These tools generate code based on some patterns derived from a big dataset; hence, AI may generate technically